

Group 3 Capstone Project

Jackson Maines, Jasmine Koh, Ethan Davis
PH 1975 L700

Table of Contents

Intended Use.....	2
Program Design.....	2
Implementation Details.....	4
1. Web Scraper Module.....	4
2. Database Module.....	5
3. Visualization Module.....	6
Results.....	8
User Guide.....	13
Contributions.....	17
References.....	17

Intended Use

We have constructed a Python-based (version 3.14.3) web scraper in Jupyter Notebook. This tool allows users to extract raw data, generate insights, and visualize trends related to scientific awards granted for mental health research in the 2025 fiscal year. It relies on the NIH RePORTER API, which provides access to project details regarding scientific awards funded by the National Institute of Health (NIH) and other federal agencies. The API serves as a data source for exploring research topics, principal investigators (PIs), funding amounts, and other related meta-data. More details can be found at <https://api.reporter.nih.gov/>. This tool has important implications in understanding public health policy and federal investment patterns in mental health research.

Program Design

We first query the NIH RePORTER API to retrieve a specific number of records on mental health research projects. The returned JSON data is converted to a pandas DataFrame and saved to a CSV file [1]. The CSV is imported into a SQLite3 database with two tables: one table for information about the project name, organization, award amount, start, and end dates and the second table for the principal investigator's names [2]. These tables are connected by each project's unique ID, allowing each PI to be connected to several research projects, each of which may be connected to other PIs in the database.

We implemented a series of SQL query functions, allowing users to search for projects by PI name, keyword, funding amount, or organization name based on user input. If users input information outside of the database, they will be informed that no such information exists. Before running a query, users must establish a connection to the SQL database with the appropriate file path.

Visualizations of key trends were performed using the Matplotlib and Seaborn libraries. These graphs can be viewed independently or within a single dashboard. As with the database, users must specify an appropriate file path to the CSV before generating graph visualizations. If the CSV file cannot be found or is corrupted, users will be prompted to enter a valid path or informed the file is corrupted, respectively. Each component of the process is modular, allowing for users to modify or build upon this tool as needed.

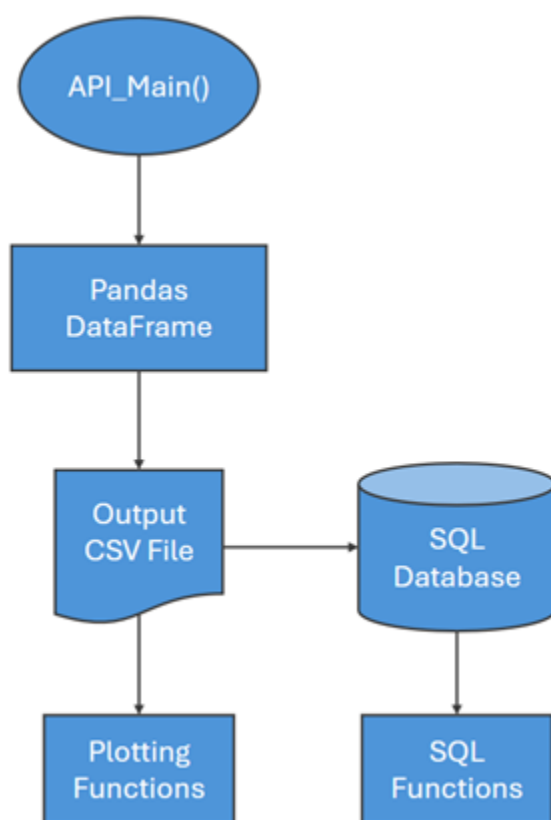


Fig 1. The general tool workflow. The NIH RePORTER API is called and converted to a pandas DataFrame which is saved locally into a CSV file. The CSV is imported into a database using SQLite3. Users may generate visualizations using the CSV or queries using the database.

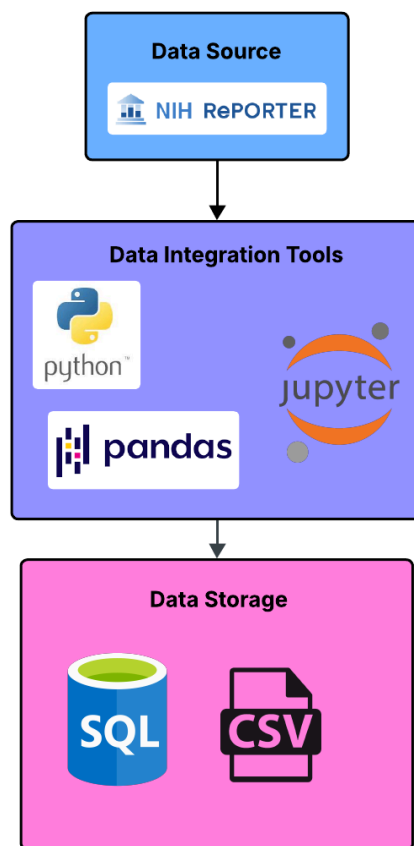


Fig 2. The tool architecture. Data is sourced from the NIH RePORTER API and manipulated using Python, pandas, and Jupyter Notebook. Data is subsequently stored in a SQL database and CSV file.

Implementation Details

1. Web Scraper Module

The process below was developed and followed to create a web scraper using the NIH RePORTER API that collects information about funding, start dates, projected end dates, organization name, principal investigator names, abstract titles, and unique meta-data for scientific research awards granted by the federal government in mental health research for the 2025 fiscal year. This information is stored into a CSV file. Functions are described and performed modularly to give users specific insight so they may make modifications as needed.

Required packages: pandas, requests, numpy

- a. API_Payload Function
 - i. Single call is made to the NIH RePORTER API with a given starting position and maximum response size of 500 using the Requests module
 - ii. Returns JSON dictionary
- b. Response_to_csv Function
 - i. Flattens JSON dictionary generated by the API_Payload function into a pandas DataFrame [3]
 - ii. Combines PI first and last names
 - iii. Converts missing values to null values using numpy [4]
 - iv. Orders PI names into list by research project and cleanly aggregates data
- c. API_Main Function
 - i. Creates empty pandas DataFrame
 - ii. Calls the API_Payload function to generate a JSON dictionary of data
 - iii. Calls the Response_to_csv Function to convert the dictionary into a temporary pandas DataFrame
 - iv. The new temporary DataFrame is concatenated to the already existing one and the starting position of the API_Payload function is updated
 - v. Clears duplicate data and updates DataFrame size
 - vi. Repeats until desired number of total data is reached (10000 calls)
 - vii. Columns are renamed and converted to the proper data type
 - viii. Function output is converted to a CSV file and saved locally

2. Database Module

The process below was developed and followed to create a database using pandas and SQLite 3. The CSV file from the web scraper module is modified and read into a SQL database for user query. We implement SQLite 3 code to allow users to search for project names and unique ID numbers based on principal investigator name, organization name, project abstract topic, or funding amount. Functions are described and performed modularly to give users specific insight so they may make modifications as needed.

Required packages: sqlite3, literal_eval from ast

- a. CSV_to_DB Function

- i. The CSV file is from the API_Main function is read in by the appropriate file path
 - ii. Rows with missing PI names are dropped
 - iii. A separate dataframe with only project number and PI names is created, separating PIs into separate rows using the `literal_eval` function from `ast` and `explode` function from `pandas` [5]
 - iv. Both dataframes are connected to the database using SQLite3's `to_sql` function [6] [7]
 - 1. Creates two tables; ProjectInfo and Names
 - v. The database tables are connected many-to-many by the primary key, `project_id`
- b. `find_pi` Function
 - i. Name of a PI is entered by user
 - ii. Connection is established to database from the `CSV_to_DB` function via appropriate file path using SQLite3
 - iii. SQL query retrieves and displays list of project names and numbers from the ProjectInfo table associated with PI from the Names table that is like user input
 - 1. SQL wildcards grant flexibility in input format, allowing users to enter first, last, full, or parts of a PI's name as a valid search
- c. `topic_search` Function
 - i. A topic or topics is entered by the user
 - 1. If multiple topics, users are prompted to separate by a single space
 - ii. Connection is established to database from the `CSV_to_DB` function via appropriate file path using SQLite3
 - iii. SQL query retrieves and displays list of project names and numbers from the ProjectInfo table where the project title is like user input
 - 1. SQL wildcards grant flexibility in input format, allowing users to enter part of a title as a valid search
- d. `greater_awards` Function
 - i. An integer amount is entered by user
 - ii. Connection is established to database from the `CSV_to_DB` function via appropriate file path using SQLite3
 - iii. SQL query retrieves and displays list of project names and numbers from the ProjectInfo table where award amount was equal to or greater than user input
- e. `org_projects` Function
 - i. An organization's name is entered by user
 - ii. Connection is established to database from the `CSV_to_DB` function via appropriate file path using SQLite3
 - iii. SQL query retrieves and displays list of project names and numbers from the ProjectInfo table where the organization name is like user input

1. SQL wildcards grant flexibility in input format, allowing users to enter first, last, full, or parts of an organization's name as a valid search

3. Visualization Module

The process below was developed and followed to create a visualization module that reads in the CSV file from the web scraper module and modifies it for visualization [8]. A set of example functions is given to the user to highlight key trends regarding popular project start and end dates, top organizations by publication count, top PIs by publication count, and the most common funding tiers. Users may view visualizations individually or all at once in a dashboard.

Required packages: matplotlib.pyplot, seaborn

- a. load_csv Function
 - i. Reads in CSV file from appropriate file path as dataframe
 - ii. Converts all columns into corresponding data types and saves as separate variable
 1. List of PI names for each project is converted from string to Python list
 2. Start and End dates are converted to datetime format
 3. Award amounts are converted to integers
 4. Remaining columns are considered strings
- b. start_month Function
 - i. Sets style to "darkgrid" using Seaborn's set_style function
 - ii. Sets palette to "PiYG" using Seaborn's set_palette function
 - iii. Creates copy of dataframe from load_csv function
 - iv. Creates new data frame, extracting months from start dates
 - v. Converts months to strings
 - vi. Creates line plot using Seaborn's lineplot function with the x-axis as the month started and the y-axis as the number of projects began that month
- c. end_month Function
 - i. Sets style to "darkgrid" using Seaborn's set_style function
 - ii. Sets palette to "PiYG" using Seaborn's set_palette function
 - iii. Creates copy of dataframe from load_csv function
 - iv. Creates new data frame, extracting months from end dates
 - v. Converts months to strings
 - vi. Creates line plot using Seaborn's lineplot function with the x-axis as the month ended and the y-axis as the number of projects ended that month

- d. top_orgs Function
 - i. Sets style to “darkgrid” using Seaborn’s set_style function
 - ii. Sets palette to “PiYG” using Seaborn’s set_palette function
 - iii. Creates copy of dataframe from load_csv function
 - iv. Creates new data frame, extracting organization names for each project
 - v. Select top 10 recurring organizations by count using pandas’ nlargest function from that data frame
 - vi. Creates horizontal bar graph using Seaborn’s barplot function with the x-axis as the count and the y-axis as the organization name
- e. award_amount Function
 - i. Sets style to “darkgrid” using Seaborn’s set_style function
 - ii. Sets palette to “PiYG” using Seaborn’s set_palette function
 - iii. Creates copy of dataframe from load_csv function
 - iv. Creates histogram using Seaborn’s histplot function with the x-axis as the award amount and the y-axis as the number of projects that received funding within that range
 - 1. The x-axis range is limited to \$7,000,000 in order to cut outliers from the visualization [9]
- f. top_pis Function
 - i. Sets style to “darkgrid” using Seaborn’s set_style function
 - ii. Sets palette to “PiYG” using Seaborn’s set_palette function
 - iii. Creates copy of dataframe from load_csv function
 - iv. Creates new data frame, separating each list of principal investigators per project into their own rows
 - v. Extract top 5 recurring PIs by grouping by PI name, sorts by count, then selecting the first 5 and save to separate data frame
 - vi. Converts PI names to title case using pandas’ str.title function
 - vii. Creates bar graph using Seaborn’s barplot function with the x-axis as the PI names and the y-axis as their publication count
 - 1. X-axis tick labels are angled 45 degrees to ensure readability
- g. show_all Function
 - i. Generates dashboard in 2 by 3 subplot grid format grid
 - ii. Flattens grid into 1D axis
 - iii. Each function detailed above is called with argument `axs[i]` to determine the placement of the graph
 - 1. Since there are only 5 graphs, one index is left empty
 - iv. Displays the dashboard

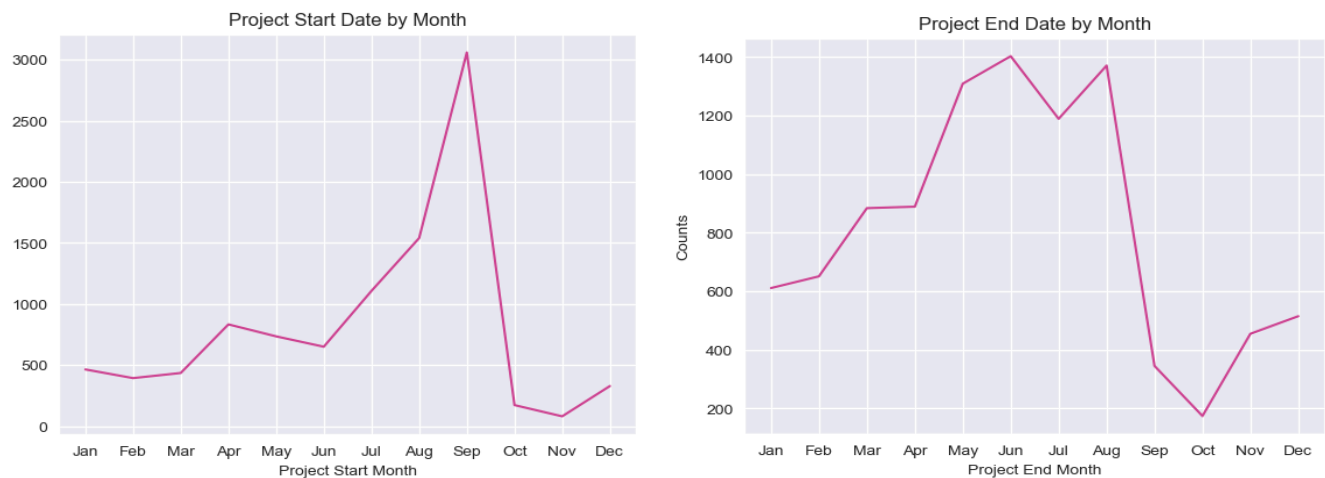
Results

An example of the SQL query function outputs can be seen below. The find_pi function was run with the surname “Vikram” being entered into the input window. Four project names and

unique IDs associated with a PI whose last, first, or partial name being “Vikram” were printed for the user.

```
Please enter the full name of a PI: Vikram
Projects related to: Vikram
Project Number: 3R01MH121632-04S1
Project Name: Improving Outcomes in Depression in Primary Care
Project Number: 5R01MH115504-05
Project Name: Implementation of evidence based facility and community interventions to reduce the treatment gap for depRESSION (IMPRESS)
Project Number: 5R01MH119086-06
Project Name: Behavioral and Neural Representations of Subjective Effort Cost
Project Number: 5K23DK138312-02
Project Name: Toward optimizing dialysis in the perioperative period
```

Five visualizations related to important information about trends in the mental health field were generated by the tool.

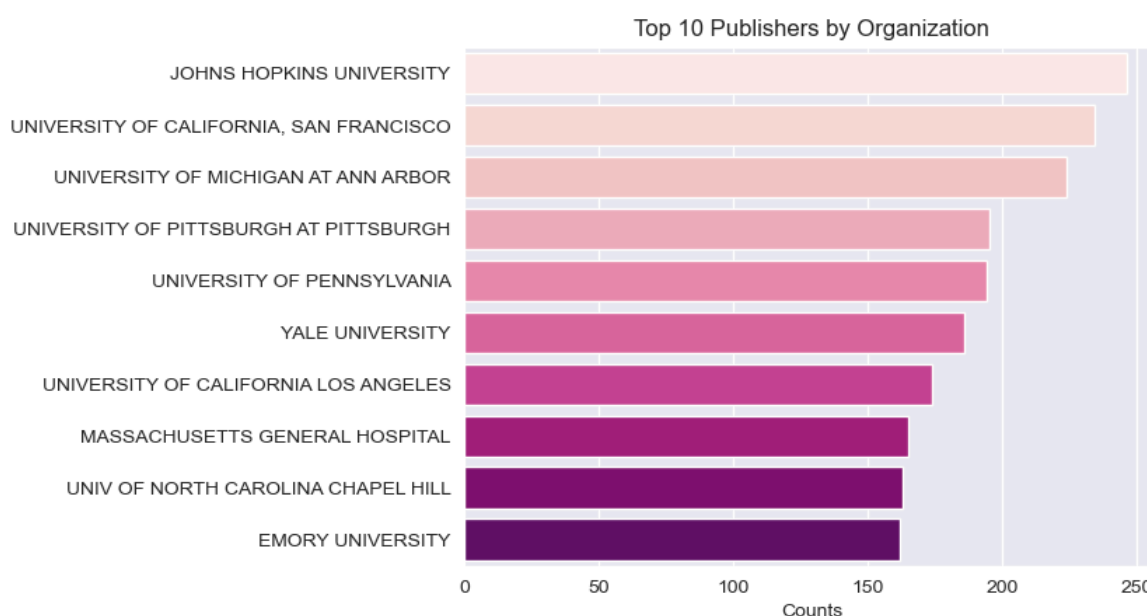


Beginning with project start months, we note that there is a particularly sharp peak in the early fall with the most common start month for research projects beginning in September at 3060 projects. This drastically drops in mid to late fall with the least common start month being November at only 80 projects. The project start rate remains relatively low throughout spring, at about 500 projects started per month, and slowly rises in the summer to about 700 projects per month until it spikes back up in August and September at 1500 projects and 3060 projects, respectively.

Comparatively, the graph for project end dates has an extended peak throughout the summer with the highest rate being 1403 projects finished in June. Similarly, the least common

month to end a project is October with only 174 projects. Project end rate slowly rises until it peaks again in May and continues throughout the summer.

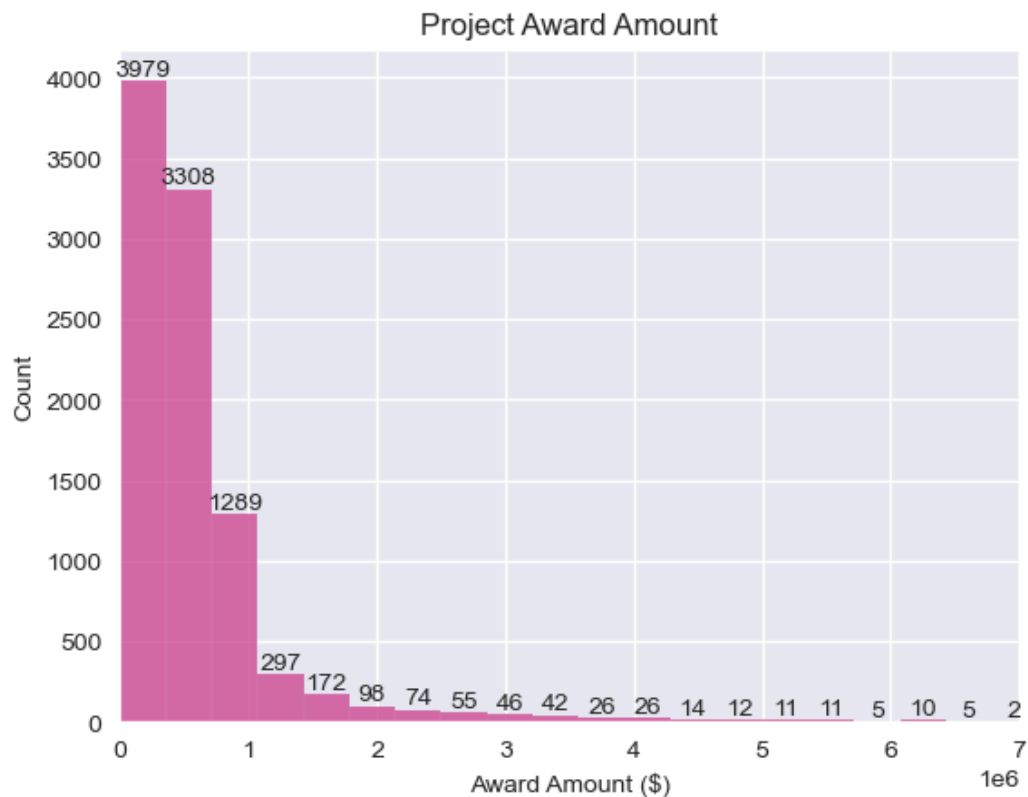
Given that most of the organizations come from academic research universities, these trends likely coincide with the start and end of academic semesters. Most projects begin at the start of the fall semester, in August and September, and finish at the end of the spring semester in May and June. Both project start and end rates heavily decline in the winter months, possibly due to lack of involvement from the holiday season.



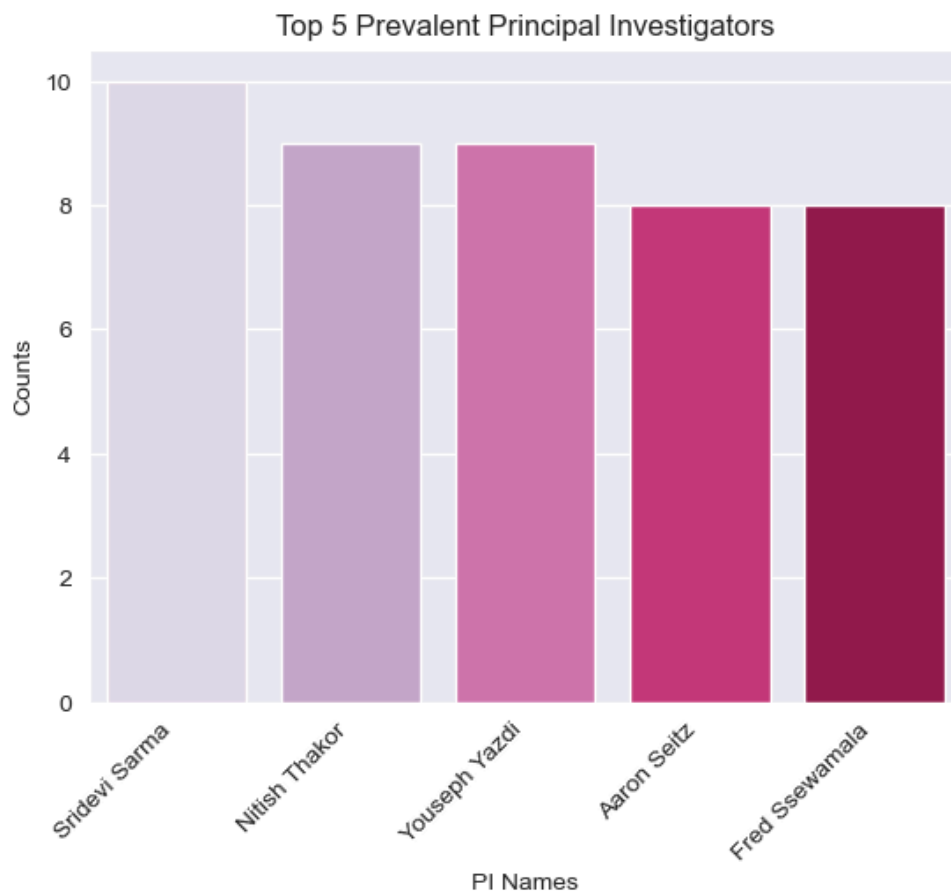
The top ten institutions that contributed to mental health research within the 2025 fiscal year were John Hopkins University (246 projects). University of California San Francisco (234) , University of Michigan at Ann Arbor (224), University of Pittsburgh (195) , University of Pennsylvania (194), Yale University (186), University of California Los Angeles (174), Massachusetts General Hospital (165), University of North Carolina Chapel Hill (163), and Emory University (162).

Although this chart does not account for the total amount of funding awarded, the data suggests that these organizations are the most consistently-funded programs and are likely the leading institutions within the mental health field. It is of note that several of these universities

are considered highly prestigious universities within highly affluent and population-dense areas. Thus, these organizations likely have the resources to maximize funds and publish high volumes of academic research.



The histogram of the award amounts reflects a right-skewed distribution with the most common funding amount being under 1 million dollars at 8481 projects. For every additional 1 million dollars, the number of projects dwindles. Outliers above 7 million dollars were removed from the visualization to reflect a more accurate spread of the funding. Around 50 awards were given above 7 million dollars with the largest value being recorded at \$357,194,999. This suggests that federal grants provide a similar level of funding to most projects but recognize and give special recognition to projects that may be critical to breakthroughs in mental health.



From this bar plot, we can identify the principal investigators that are funded the most often. Given that only 83% of PIs currently lead or had led one funded project within the 2025 fiscal year (9621 PI names occurred once in the data set), the output of these PIs is far above average. While this may not necessarily be reflective of leadership within the field, as it does not reflect their level of funding nor impact, it does indicate that these PIs receive more consistent funding and likely have a higher impact on the field than the average due to their number of projects. For example, Sridevi Sarma, the most prevalent PI in the data set, received a total funding of \$1,196,697 for a single project. Three of the top five PIs, Sridevi Sarma, Nitish Thakor, and Youseph Yazdi, are from John Hopkins University. From previous analyses, we know that John Hopkins is a highly prestigious research institution. This combination of

university prestige, successful track record, and impactful research outcomes may result in federal research grants being more readily awarded to these PIs.

User Guide

A simplified user guide is written below to assist users with setting up this tool in Jupyter Notebook. An explanation of each of the important files within the folder follows.

Setting up Virtual Environment (VS Code)

1. Download the “phIDS-Project-main” folder and set this folder to your working directory in VS Code
2. Install MiniForge for your OS
3. In the MiniForge terminal, run: `conda env create -f Project_Env.yml`
`conda activate PH1975Project`
4. Once finished, open the “Notebook_Demo.ipynb” file in VS Code
5. Ensure the working kernel is set to “PH1975Project”

SQL Functions General Workflow

1. Run `import Functions as fuc`
 - a. If an error appears, refer to the “Setting Up Virtual Environment” section of the User Guide
2. Run the code block containing `API_DF = API_Main(iterating_offset = 0, total_calls = 10000, api_limit = 500)`
 - a. This will automatically run `API_DF.to_csv('NIH_CSV.csv', index = False)`
 - b. These will create a local CSV file of over 10,000 rows of information about scientific awards.
3. To run SQL functions, first run the code block containing `fuc.CSV_to_DB(path_to_csv = 'NIH_CSV.csv', path_to_db = 'NIH_DB.db')`
 - a. Ensure the CSV file is in your desired working directory
 - b. All SQL functions can be run with the argument specified to *path_to_db* in the last step
4. To run visualizations, ensure the CSV file is in your working directory.
 - a. All visualization functions can be run with the argument specified to *path_to_csv* in step 2

Project_Env.yml

Contains a custom virtual environment.

Channels: conda-forge

Dependencies: Python (3.14.3), request, matplotlib, pandas, seaborn, sqlite3, ipykernel

Functions.py

Contains all functions needed to produce intended data.

- o API_Payload: queries the NIH RePORTER API with hard coded payload. Helper function to API_Main.

§ Arguments:

offset: Starting position of API query

payload_limit: Number of responses a single query that the NIH API can handle (Maximum of 500).

§ Returns:

api_dict: Dictionary of API responses.

- o Response_to_csv: takes api_dict as input and converts it to pandas dataframe. Helper function to API_Main.

§ Arguments:

data: The dictionary from API_Payload, api_dict

§ Returns:

temp_df: Dictionary flattened into pandas dataframe.

- o API_Main: the parent function to API_Payload and Response_to_csv, function loops queries until number of wanted outputs is achieved.

§ Arguments:

iterating_offset: Starting position of API query

total_calls: The number of total wanted responses from the API.

api_limit: Number of responses a single query that the NIH API can handle (Maximum of 500).

§ Returns:

scrapping_df: Full pandas dataframe of all API queries.

- o CSV_to_DB: creates SQL database from NIH CSV made from API_Main.

§ Arguments:

path_to_csv: Path to csv created from API_Main.

path_to_db: Desired file path of SQL Database.

- o find_pi: finds projects related to user inputted Principal Investigator. Prints project number and name.

§ Arguments:

path_to_db: The file path to the SQL database specified in CSV_to_DB.

- o topic_search: finds projects related to an inputted topic. Prints project number and name for each topic. Topics are user inputted and split by space.

§ Arguments:

path_to_db: The file path to the SQL database specified in CSV_to_DB.

- o greater_award: finds projects with more award amount than user inputted value. Prints project number, name, and award total.

§ Arguments:

path_to_db: The path to the SQL database specified in CSV_to_DB.

- o org_projects: finds projects related to a user inputted organization. Prints project number and name.

§ Arguments:

path_to_db: The path to the SQL database specified in CSV_to_DB.

Demo_Notebook.ipynb

A Jupyter Notebook showcasing use of all above functions. Each code chunk can be run to validate all results or for user exploration.

Full_Data.csv

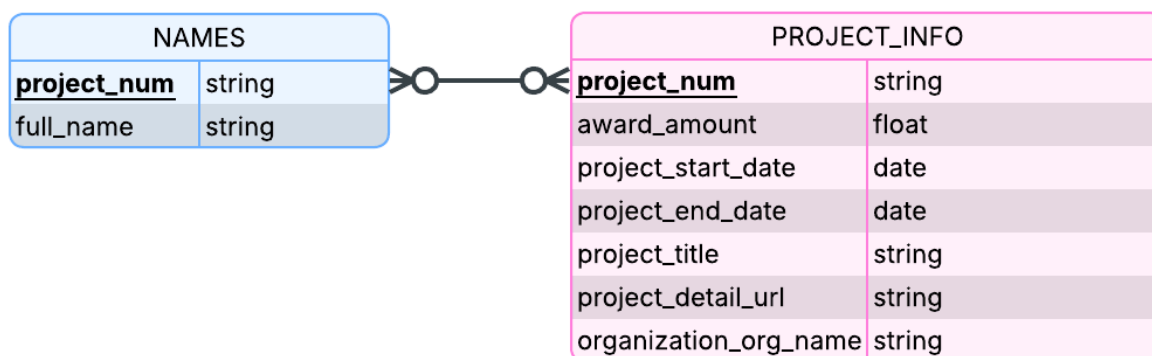
A CSV file containing information on over 10,000 scientific awards scraped from the NIH RePORTER API using this tool.

Column Descriptions:

- o project_num: Unique project number given to each award.
- o pi_names: List of principal investigators working on the awarded project.
- o award_amount: The amount awarded to the organization.
- o project_start_date: The year-month-day the project was started.
- o project_end_date: The year-month-day the project ended.
- o project_title: The title of the research project.
- o project_detail_url: The direct link to the NIH project page.
- o organization: Name of the funded organization.

Full_Database.db

A SQLite database of information on over 10,000 scientific awards scraped from the NIH RePORTER API using this tool. Imported from the previous CSV file. The database is composed of two tables: ProjectInfo and Names. Their relationship is visualized by the Entity-Relationship Diagram below.



Contributions

Jackson Maines: main code base development and implementation for scraper module and database module

Jasmine Koh: main code base development and implementation for visualization module

Ethan Davis: primary author for project report and code review for previous sections

References

[1] McKinney, W., “Data Structures for Statistical Computing in Python,” *Proc. 9th Python in Science Conf.*, pp. 51–56, 2010. doi: 10.22008/scipy-proc-2010-mckinney.

[2] R. D. Hipp, “SQLite,” 2020. [Online]. Available: <https://www.sqlite.org/index.html>

[3] GeeksforGeeks, “Python Pandas Flatten nested JSON,” *GeeksforGeeks*, Mar. 16, 2022. <https://www.geeksforgeeks.org/python/python-pandas-flatten-nested-json/>

[4] C. R. Harris, K. J. Millman, S. J. van der Walt *et al.*, “Array programming with NumPy,” *Nature*, vol. 585, pp. 357–362, 2020. doi: 10.1038/s41586-020-2649-2.

[5] Z. Bobbitt, “How to Use the Pandas explode() Function (With Examples),” *Statology*, Dec. 30, 2021. <https://www.statology.org/pandas-explode/>

[6] GeeksforGeeks, “Python SQLite Cursor Object,” *GeeksforGeeks*, May 05, 2021. <https://www.geeksforgeeks.org/python/python-sqlite-cursor-object/>

[7] GeeksforGeeks, “Creating a sqlite database from CSV with Python,” *GeeksforGeeks*, Dec. 25, 2020. <https://www.geeksforgeeks.org/python/creating-a-sqlite-database-from-csv-with-python/>

[8] A. Kumar, “Creating Stunning Dashboards with Matplotlib and Seaborn,” *Medium*, Feb. 25, 2025. <https://medium.com/@amitt2015/creating-stunning-dashboards-with-matplotlib-and-seaborn-8b9383b6e904>

[9] J. Frost, "Guidelines for Removing and Handling Outliers in Data," *Statistics By Jim*, 2019.
<https://statisticsbyjim.com/basics/remove-outliers/>